

NATURAL LANGUAGE PROCESSING

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Sarah Malaeb
2025-26

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

L'IA générative a révolutionné le NLP en permettant aux modèles de produire un texte cohérent, conscient du contexte et spécifique au domaine.

Exemple : Cette transformation est particulièrement importante dans les domaines numériques, l'IA/ML et l'extraction de compétences, où les connaissances sont dynamiques et très spécialisées.

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Large Language Models (LLMs)

Pré-entraînés sur d'énormes corpus (manuels, articles de recherche, sites web). Capables d'apprentissage few-shot et zero-shot, réduisant le besoin de données étiquetées.

Exemples de LLM : GPT-4, LLaMA, Mistral, Claude

Exemple : Prompt : « List emerging Data Science frameworks for 2025. »

Sortie : “Frameworks Python ML (PyCaret 3.0, LangChain, mises à jour HuggingFace Transformers).”

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Compréhension contextuelle

Les modèles traditionnels (Word2Vec, GloVe) produisent des embeddings statiques.

Les modèles d'IA générative capturent des embeddings contextuels, la syntaxe, la sémantique et les dépendances entre mots.

Exemple :

Phrase A : « Data scientists analyze datasets. »

Phrase B : « Datasets are investigated by data engineers. »

Les modèles génératifs comprennent que ces phrases sont sémantiquement identiques et peuvent générer une liste unifiée de compétences.

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Intégration et raisonnement des connaissances

L'IA générative peut combiner des informations provenant de plusieurs sources et produire des sorties factuelles et structurées.

Exemple en extraction de compétences :

Input : « Extract AI/ML skills from 3 recent research papers »

Sortie : Ensemble de compétences consolidé avec références aux articles, incluant des outils émergents comme PyTorch Lightning, Ray, ou Mistral.

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Résumé et synthèse

Les LLMs peuvent résumer de longs documents techniques en sorties digestes.

Exemple : Résumer un rapport de 50 pages sur « AI-based predictive analytics in healthcare » en :

- Compétences : Python, Algorithmes ML, Évaluation de modèles
- Outils : TensorFlow, PyTorch
- Techniques : Prédiction de séries temporelles, Détection d'anomalies

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Génération de code et Data Science

L'IA générative prend désormais en charge la génération de code, de notebooks et de templates de workflows.

Exemples :

- Générer un script Python pour le prétraitement de datasets NLP
- Auto-générer des pipelines de feature engineering pour des datasets tabulaires
- Suggérer des workflows de visualisation de données pour des expériences ML

AVANCÉES EN NLP ASSISTÉ PAR L'IA GÉNÉRATIVE

Adaptation au domaine

L'IA générative peut être fine-tunée sur des corpus spécifiques à un domaine (finance, santé, recherche IA).

Sortie : connaissance consciente et contextuellement pertinente.

Exemple : Générer des données pertinentes pour un mapping de compétences numériques spécifique à un domaine.

IMPLICATIONS DES APPLICATIONS NLP

NLP Task	Generative AI Advancement	Digital Domain Example
QA / Information Retrieval	Context-aware, multi-document answers	Extract ML competencies from research papers
Summarization	Condenses long technical reports	Summarize AI curriculum for skill mapping
Knowledge Synthesis	Combine multiple sources	Build Data Science skill ontology
Text Generation	Create domain-specific content	Generate job descriptions or skill frameworks
Code Generation	Translate instructions to code	Auto-generate ML preprocessing pipelines

LIMITATIONS DE L'IA GENERATIVE AI EN NLP

Knowledge Cutoff : Les modèles sont entraînés sur des données jusqu'à une certaine date → risque de manquer de nouveaux outils, compétences ou frameworks.

- Exemple : GPT-4 peut ne pas connaître PyCaret 3.0 ou les mises à jour Mistral 2025.

Hallucinations : Peut générer des sorties plausibles mais incorrectes.

- Exemple : Invention d'une compétence Data Science inexistante.

Mémoire limitée : Les grandes bases de connaissances ne peuvent pas être entièrement encodées dans les poids du modèle.

Raisonnement multi-document : Difficile de consolider l'information provenant de plusieurs sources.

- Exemple : Extraire des compétences à partir de 10 articles de recherche et offres d'emploi.

Adaptation spécifique au domaine : Nécessite un fine-tuning pour des domaines de niche (IA, ML, cybersécurité).

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Taille des modèles et coût computationnel

Les LLM nécessitent d'énormes ressources mémoire et de calcul.

- Exemple : GPT-4 ou Mistral Medium 3 nécessite des GPU/TPU spécialisés.

L'entraînement ou le fine-tuning peut être **coûteux** pour les organisations.

Qu'est-ce que le fine-tuning d'un LLM ?

Le fine-tuning est le processus d'adaptation d'un LLM pré-entraîné (comme GPT, BERT ou Mistral) à une tâche ou un domaine spécifique en l'entraînant davantage sur des données spécifiques à la tâche.

LLM pré-entraîné : apprend les motifs linguistiques généraux, la grammaire et des connaissances larges sur le monde.

LLM fine-tuné : optimisé pour produire de meilleurs résultats pour une application spécifique (ex. : extraction de compétences IA, génération de code ou analyse de textes médicaux).

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Défis du fine-tuning

Exigences en données : le fine-tuning nécessite de grands jeux de données spécifiques au domaine et de haute qualité.

Oubli catastrophique : les modèles peuvent perdre des connaissances générales lorsqu'ils s'adaptent à un nouveau domaine.

Complexité des hyperparamètres : optimiser le taux d'apprentissage, la taille des batches et les couches est complexe.

- **Exemple** : Fine-tuner GPT pour extraire des compétences IA/Data Science à partir d'articles de recherche, offres d'emploi et documentation interne peut être difficile en raison de la diversité des données.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Limite des connaissances (Knowledge Cutoff)

Les modèles pré-entraînés ne connaissent que les informations disponibles jusqu'à une certaine date.

Ils ne peuvent pas intégrer automatiquement des frameworks, outils ou standards émergents.

- **Exemple :** de nouvelles librairies comme PyCaret 3.0 ou les modèles Mistral peuvent ne pas être reconnus.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Hallucinations et erreurs factuelles

L'IA générative peut produire des sorties **plausibles** (raisonnables et possibles) mais incorrectes.

Le risque est élevé pour l'extraction de données et les rapports spécifiques à un domaine.

- **Exemple :** Inventer une compétence IA/ML inexistante lors d'un mapping de compétences.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Raisonnement multi-document

Difficulté à synthétiser les informations provenant de **multiples sources**.

- **Exemple** : Important pour résumer des curricula, consolider des compétences à partir de plus de 10 articles de recherche ou d'offres d'emploi.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Adaptation au domaine

Les modèles généraux sous-performent souvent sur des domaines spécialisés sans fine-tuning.

- **Exemple** : S'adapter aux domaines IA/Data Science, finance ou santé nécessite un effort significatif.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Limites de la fenêtre contextuelle

Les LLM ne peuvent traiter qu'un **nombre limité de tokens** à la fois.

Les grandes bases de connaissances ou les longs documents peuvent être tronqués, entraînant la perte d'informations cruciales.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Vie privée et conformité

Les modèles peuvent **mémoriser des informations sensibles** provenant des données d'entraînement.

- **Exemple** : important lors de l'utilisation de documents internes, rapports d'entreprise ou données personnelles pour l'extraction de compétences.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Difficultés d'évaluation

Les métriques standard (BLEU, ROUGE) peuvent ne pas refléter la justesse spécifique au domaine.

Nécessite une **validation experte**, par exemple pour l'extraction de compétences numériques.

Autres métriques associées :

- **METEOR** : prend en compte les synonymes, le stemming et l'alignement entre le texte de référence et le texte généré.
- **BERTScore** : utilise des embeddings contextuels (BERT) pour mesurer la similarité sémantique entre le texte généré et le texte de référence.
- **Évaluation humaine** : particulièrement importante pour l'extraction de compétences spécifiques au domaine ou les connaissances numériques, car les métriques automatisées peuvent ne pas capturer la justesse sémantique.

DÉFIS DE L'IA GÉNÉRATIVE EN NLP

Difficultés d'évaluation

<u>BLEU (Bilingual Evaluation Understudy)</u> Purpose: Evaluate machine translation or generated text by comparing overlap of n-grams with reference text. How it works: - Count matching n-grams (sequences of words) between generated and reference text. - Apply a brevity penalty if generated text is shorter than reference. Score Range: 0 (no overlap) → 1 (perfect match)	<u>ROUGE (Recall-Oriented Understudy for Gisting Evaluation)</u> Purpose: Evaluate summarization or generative models by measuring recall of overlapping n-grams. Variants: - ROUGE-N: n-gram overlap (e.g., ROUGE-1 = unigrams, ROUGE-2 = bigrams) - ROUGE-L: Longest Common Subsequence (LCS) - ROUGE-S: Skip-bigram co-occurrence Score Range: 0 → 1, higher = better overlap with reference
<u>Limitations:</u> Sensitive to exact wording; synonyms or rephrasing are penalized. Not ideal for free-form text summarization such as competency extraction where word order varies.	<u>Advantages:</u> More robust than BLEU for summarization and free-form outputs. <u>Limitations:</u> It cannot capture synonyms or paraphrases. It focuses on words/phrases, not meaning or context. Not ideal for short outputs: Small variations in wording can dramatically affect the score. It evaluates text sequences, not structured outputs like tables, ontologies, ...
<u>Example (Digital Domain):</u> Reference competency list: ["Python", "PyTorch", "Machine Learning"] Generated output: ["Python", "Machine Learning", "TensorFlow"] BLEU measures the overlap of terms like "Python" and "Machine Learning".	<u>Example (Digital Domain):</u> Reference summary of AI skills: "Python, PyTorch, ML algorithms, Data Visualization" Generated summary: "Python, Machine Learning, PyTorch, Visualization" ROUGE captures the recall of important keywords like "Python" and "PyTorch" even if order differs.

QU'EN EST-IL DE RAG ?

Même si les modèles d'IA générative sont puissants, ils ne peuvent pas accéder de manière dynamique aux informations les plus récentes ni à de très grandes bases de connaissances.

C'est là qu'intervient le Retrieval-Augmented Generation (RAG).

RAG combine les forces des LLM (compréhension contextuelle, génération) avec la récupération d'informations depuis des bases de connaissances externes.

Exemple : Cela permet une extraction précise, à jour et spécifique au domaine des compétences, tout en contournant les limites de l'IA générative seule.

QU'EN EST-IL DE RAG ?

Les modèles NLP génératifs traditionnels (GPT, variantes BERT, T5, etc.) génèrent du texte uniquement à partir de leurs données d'entraînement. Cependant :

Défis:

- Limites de mémoire : les modèles génératifs ne peuvent pas mémoriser toutes les connaissances spécifiques au domaine.
- Obsolescence des connaissances : les modèles pré-entraînés peuvent contenir des informations dépassées.
- Hallucinations : les modèles génératifs purs peuvent produire des sorties plausibles mais incorrectes.

Solution:

RAG combine la **récupération** (accès à des connaissances externes) et la **génération** (production de texte), améliorant la précision, l'ancrage et la validité factuelle.

QU'EN EST-IL DE RAG ?

Exemple 1:

Question: “Who won the Nobel Peace Prize in 2024?”

- GPT seul pourrait deviner en se basant sur ses données antérieures.
- RAG récupère les informations les plus récentes à partir de sources externes (Wikipedia, rapports d'entreprise) avant de générer la réponse.

QU'EN EST-IL DE RAG ?

Exemple 2:

Problème dans le domaine numérique :

- Les organisations veulent souvent extraire des compétences, savoirs et unités de connaissance à partir de multiples sources (offres d'emploi, articles de recherche, documents internes, etc.).
- Les modèles NLP traditionnels peuvent avoir des difficultés à fournir des extractions précises, à jour et spécifiques au contexte.

Challenges:

- Connaissances vastes et dynamiques (l'IA/Data Science évolue rapidement)
- Les modèles génératifs purs peuvent halluciner des compétences ou outils obsolètes
- Capacité limitée à récupérer des informations spécifiques au domaine.

Solution:

RAG (Retrieval-Augmented Generation) : combine la récupération de documents pertinents avec la génération de sorties structurées.

Task:

Extract competencies for a “Machine Learning Engineer” from multiple sources.

- RAG récupère les définitions des compétences en ML à partir d'articles de recherche récents, de curricula en ligne ou de bibliothèques de compétences d'entreprise.

Le générateur produit une **liste consolidée de compétences**, telles que :

- Supervised/Unsupervised Learning
- Feature Engineering
- Data Preprocessing & Cleaning
- Déploiement de pipelines ML

REVUE DES MODÈLES NLP GÉNÉRATIFS

Modèles Encoder-Decoder : (T5, BART)

- Encode le texte d'entrée et génère la sortie.

Modèles Encodeur seul : (BERT, RoBERTa, DistilBERT)

- Encode l'entrée en représentations contextuelles.
- Principalement utilisés pour les tâches de compréhension : classification, reconnaissance d'entités nommées (NER), similarité sémantique, génération d'embeddings.

Modèles Décodeur seul : (GPT)

- Prédit le token suivant à partir de l'entrée et du contexte.

Limites pour QA factuelle / tâches spécifiques au domaine :

- Connaissances figées
- Modèles volumineux → inférence coûteuse

RAG améliore ces modèles en incorporant dynamiquement des connaissances externes.

RETRIEVAL-AUGMENTED GENERATION (RAG)

RAG est un paradigme NLP où un modèle génératif est complété par un **retriever** qui récupère des documents pertinents à partir d'une base de connaissances externe.

Flux : Question d'entrée → Retriever récupère les documents pertinents → Générateur produit la réponse en se basant sur les documents récupérés

- **Retriever** : moteur de recherche / similarité d'embeddings / base de vecteurs denses
- **Generator** : généralement un modèle seq2seq (ex. BART, T5) ou un modèle décodeur seul (GPT)

Avantages :

- Exactitude factuelle
- Adaptation au domaine sans réentraîner de grands modèles
- Réduction des hallucinations

ARCHITECTURE ET COMPOSANTS DE RAG

RAG = Retriever + Generator

Retriever : trouve les documents sémantiquement pertinents dans une base de connaissances (offres d'emploi, articles académiques, rapports internes)

Generator : produit des sorties structurées à partir du contexte récupéré

Pipeline simplifié :

Query → Retriever → récupération des documents pertinents → Generator → Réponse / sortie structurée

ARCHITECTURE ET COMPOSANTS DE RAG

Pipeline de haut niveau

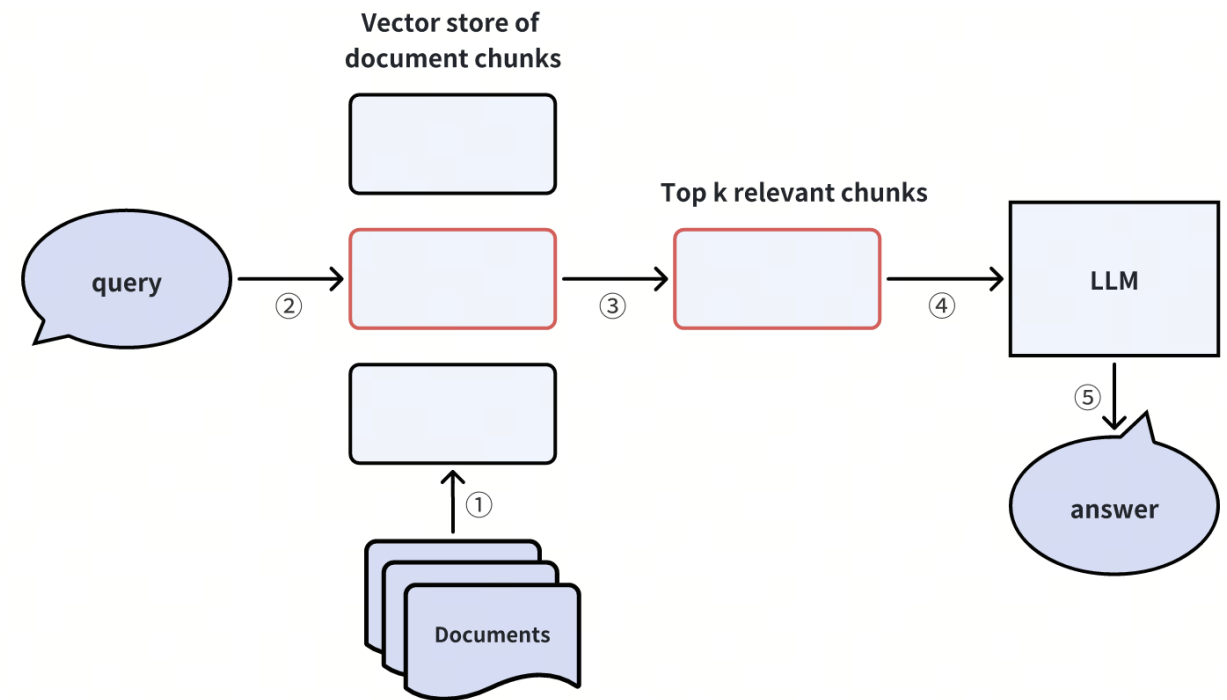
Input Query : l'utilisateur pose une question

Récupération de documents :

- Recherche de similarité basée sur des embeddings (FAISS, Milvus, Pinecone)
- Recherche traditionnelle par mots-clés (BM25)

Augmentation du contexte : les documents récupérés sont concaténés à l'entrée

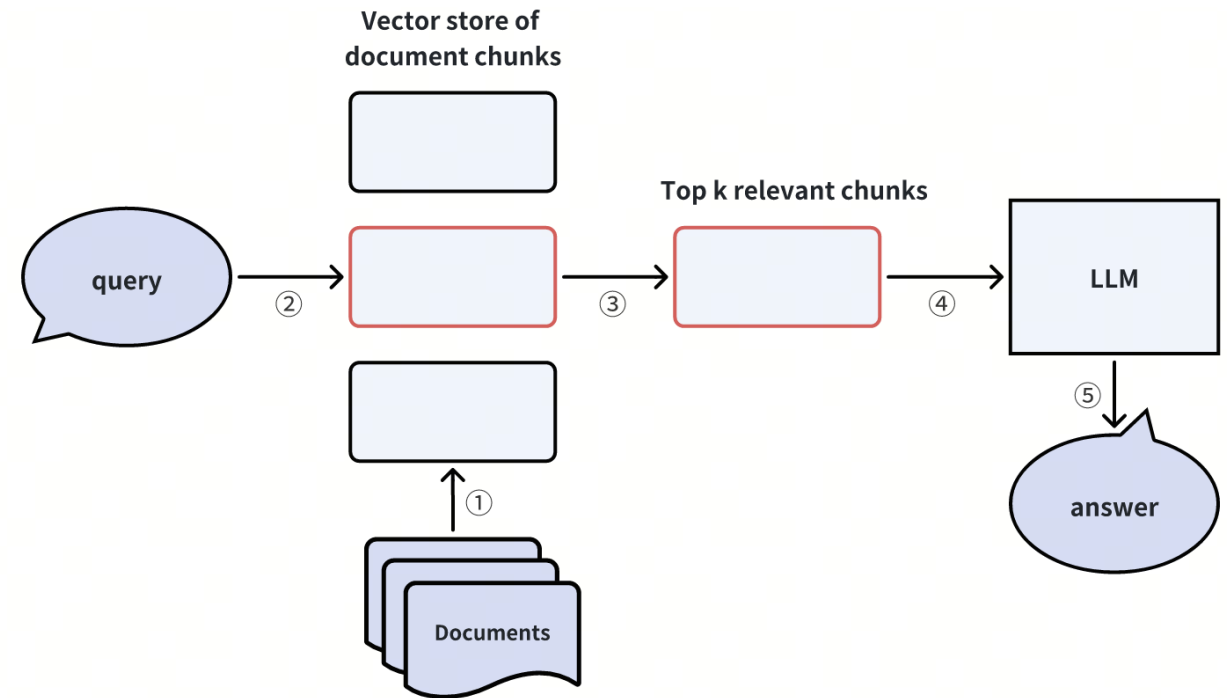
Génération : le modèle génère la réponse en se basant sur le contexte récupéré



ARCHITECTURE ET COMPOSANTS DE RAG

Exemple

- User Query: "Extract Data Science competencies for ML Engineer role"
- Retriever → Knowledge Base of Job Postings, Papers, Skill Frameworks
- Top 500 Relevant Chunks from Top 5 Relevant Docs
- Generator (BART/T5)
- Structured Output: ["Python", "ML Algorithms", "Data Visualization", "Model Deployment"]



ARCHITECTURE ET COMPOSANTS DE RAG

Types de Retriever

Sparse Retrieval : Bag-of-Words ou TF-IDF / BM25

- Rapide, interprétable, mais moins sensible à la sémantique

Dense Retrieval : basé sur des embeddings (sentence-transformers, BERT, SBERT)

- Capture la similarité sémantique

ARCHITECTURE ET COMPOSANTS DE RAG

Modèles Générateurs

Modèle génératif pré-entraîné (BART, T5, GPT)

Fine-tuné sur des données spécifiques à la tâche

Input : [query + contexte récupéré]

Output : réponse / résumé / classification

USE CASES ET APPLICATIONS

- **Question Answering** : Wikipedia QA, QA en entreprise
- **Customer Support** : automatiser des réponses via la base de connaissances de l'entreprise
- **NLP légal / médical** : récupération dans des cas juridiques, littérature médicale
- **Génération de code** : récupération de snippets pour une génération de code contextuelle
- **Résumé de documents** : récupérer des documents liés et résumer collectivement

EXEMPLE IMPLEMENTATION (PYTHON / PSEUDO-CODE)

Prérequis :

- transformers (Hugging Face)
- sentence-transformers
- FAISS pour la recherche dense : FAISS est le moteur qui récupère les passages de texte pertinents à partir de votre base de connaissances (kb_docs.csv) pour une requête donnée.

Notes:

- passages_path indique le chemin vers la base de connaissances
- Pré-calculer les embeddings pour accélérer la récupération
- Peut intégrer une base de vecteurs externe

What is FAISS?

FAISS stands for **Facebook AI Similarity Search**.

- It is an **open-source library by Meta (Facebook AI)** for efficient **similarity search and clustering of dense vectors**.
- Used for **retrieval in RAG pipelines**: when you embed documents into vectors (using Sentence-BERT, etc.), FAISS lets you quickly **search the most similar vectors** to a query.

Advantages:

- Scales to millions of vectors.
- Supports approximate nearest neighbor (ANN) search for fast results.
- Optimized for GPU and CPU.

```
from sentence_transformers import SentenceTransformer
from transformers import RagTokenizer, RagRetriever, RagSequenceForGeneration
import faiss

# Step 1: Load the retriever and tokenizer
tokenizer = RagTokenizer.from_pretrained("facebook/rag-sequence-nq")
retriever = RagRetriever.from_pretrained("facebook/rag-sequence-nq", index_name="custom", passages_path="kb_docs.csv")

# Step 2: Load RAG model
model = RagSequenceForGeneration.from_pretrained("facebook/rag-sequence-nq", retriever=retriever)

# Step 3: Query input
query = "Who won the Nobel Peace Prize in 2024?"
inputs = tokenizer(query, return_tensors="pt")

# Step 4: Generate answer
outputs = model.generate(**inputs)
answer = tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]
print(answer)
```



QUIZ